FinRelief – AI : AI - Powered Debt Relief & Financial Recovery Platform

Project Description:

FinRelief AI is an AI-Powered Debt Relief and Financial Recovery Platform designed to help individuals analyze their financial condition, manage debt effectively, and make informed financial decisions. The platform combines Artificial Intelligence with financial analytics to generate personalized debt settlement recommendations, provide concise financial guidance through an AI chatbot, generate professional negotiation letters for lenders, and monitor overall financial health using an interactive dashboard. It also maintains a financial history for tracking user progress and generates comprehensive PDF reports containing financial summaries, AI recommendations, and negotiation letters.

Built with a modern full-stack architecture, the application features a **React.js** frontend for a responsive and user-friendly interface, a **FastAPI** backend for high-performance API services, **SQLite** with **SQLAlchemy ORM** for efficient data management, and the **Google Gemini AI API** for intelligent financial assistance. Security is enhanced through **bcrypt password hashing**, strong password validation, secure authentication, and password reset functionality. By integrating AI-driven insights with financial analysis and automated document generation, FinRelief AI empowers users to improve financial stability, negotiate effectively with lenders, and confidently manage their debt recovery journey.

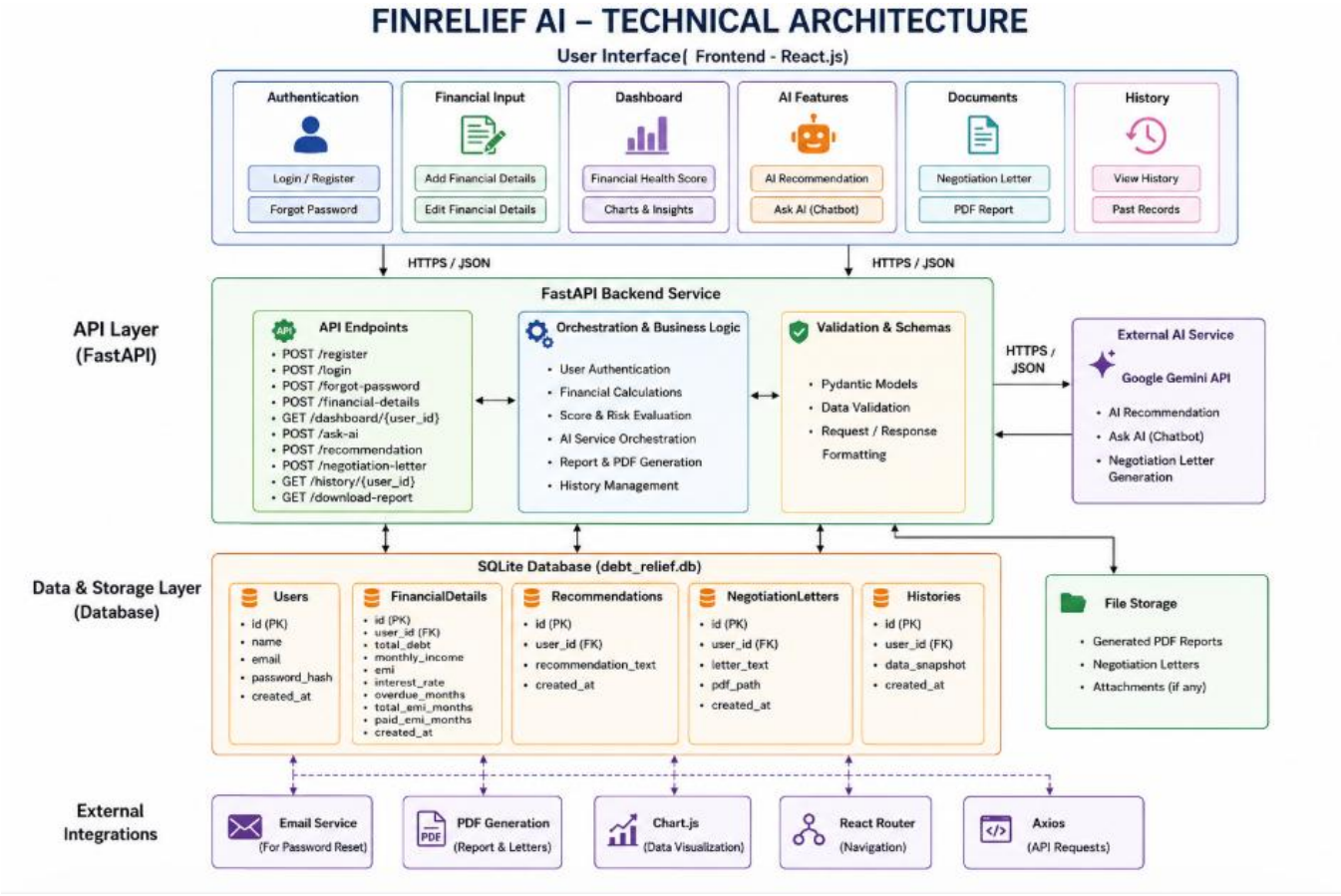
Scenarios:

Scenario 1: A user is experiencing financial stress due to multiple outstanding loans and overdue EMIs. They enter details such as total debt, monthly income, EMI amount, interest rate, overdue months, and EMIs paid. The platform analyzes their financial profile, automatically calculates the remaining EMI duration, evaluates their financial health score, and generates personalized AI-powered debt settlement recommendations to improve their financial stability.

Scenario 2: A user wants guidance on managing their finances more effectively. They use the **Ask AI** feature to ask questions such as *"How can I reduce my debt?"* or *"How can I improve my savings?"* The AI chatbot provides concise, easy-to-understand financial advice in numbered points, enabling users to make informed financial decisions without reading lengthy explanations.

Scenario 3: A user needs to negotiate with their lender for a repayment extension or debt settlement. The platform generates a professional AI-powered negotiation letter based on the user's financial information. The user can then download a comprehensive PDF report containing their financial summary, financial health score, AI recommendations, and negotiation letter. Additionally, the **History** feature allows users to review previously submitted financial records, monitor their financial progress over time, and compare improvements across multiple sessions.

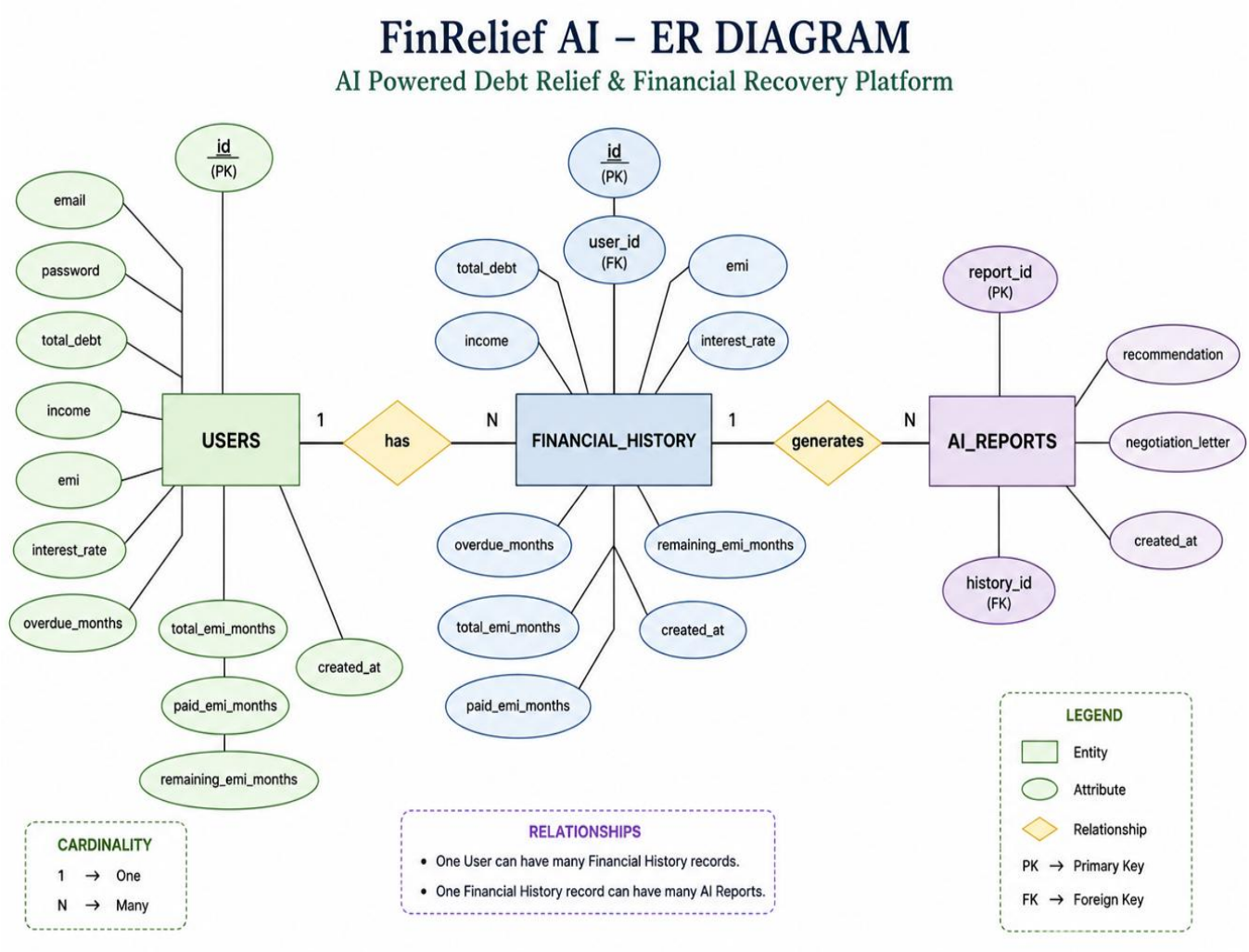
Technical Architecture:



Description:

FinRelief AI follows a modular three-tier architecture to deliver secure, AI-powered financial assistance and debt management solutions. The frontend is built with **React.js**, providing a responsive and user-friendly interface for authentication, financial data entry, dashboard visualization, AI interactions, and report generation. The **FastAPI** backend handles API routing, business logic, financial calculations, request validation using **Pydantic**, and secure user authentication with **bcrypt** password hashing. It integrates seamlessly with the **Google Gemini AI API** to generate personalized financial recommendations, answer user queries through an AI chatbot, and create professional debt negotiation letters. Financial data and user history are persistently stored in an **SQLite** database using **SQLAlchemy ORM**, while PDF reports are generated dynamically to provide comprehensive financial summaries and AI-generated insights. This architecture ensures scalability, maintainability, security, and efficient AI-driven financial analysis.

ER Diagram:



Description:

The **Entity Relationship Diagram (ERD)** for **FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform** illustrates the logical database design by defining the core entities, their attributes, primary and foreign keys, and the relationships between them. The **Users** entity stores authentication credentials and financial profile details, **Financial History** maintains historical debt and repayment records for each user, and **AI Reports** stores AI-generated debt settlement recommendations and negotiation letters linked to financial history. This structured database design ensures efficient data management, minimizes redundancy through normalization, maintains referential integrity, and enables accurate financial analysis, AI-driven recommendations, report generation, and historical tracking across the platform.

Pre-requisites:

- **Python Programming Proficiency:** Knowledge of Python 3.10+ and virtual environment (venv) management.
- **FastAPI Framework:** Understanding of FastAPI for building REST APIs, routing, dependency injection, and request validation using Pydantic.
- **React.js Fundamentals:** Familiarity with React.js components, hooks, React Router, and state management.
- **HTML, CSS & JavaScript:** Basic knowledge of frontend development for creating responsive user interfaces.
- **SQLite & SQLAlchemy ORM:** Understanding of relational databases, SQL queries, and ORM-based database operations.
- **Google Gemini AI API:** Familiarity with integrating Generative AI models for financial recommendations, chatbot responses, and negotiation letter generation.
- **Axios & REST APIs:** Knowledge of sending HTTP requests between the React frontend and FastAPI backend.
- **Password Security:** Understanding of bcrypt hashing for secure password storage and authentication.
- **Git & GitHub:** Basic version control knowledge for source code management and collaboration.
- **PDF Generation:** Familiarity with React PDF generation libraries for creating downloadable financial reports.

Project Workflow:

Milestone 1: Project Setup & System Architecture

- **Activity 1.1:** Set up the development environment by creating a Python virtual environment and installing required dependencies such as FastAPI, React.js, SQLite, SQLAlchemy, Axios, bcrypt, and Google Gemini AI SDK.
- **Activity 1.2:** Design the overall system architecture by separating the application into Frontend (React), Backend (FastAPI), AI Services (Gemini API), and Database (SQLite).
- **Activity 1.3:** Configure SQLAlchemy ORM models and establish the SQLite database for user authentication and financial records.
- **Activity 1.4:** Create API routing structure for authentication, financial management, AI recommendation generation, negotiation letter generation, and history management.

Milestone 2: User Authentication & Security

- **Activity 2.1:** Develop secure user registration with email validation and strong password policy enforcement.
- **Activity 2.2:** Implement secure login functionality using bcrypt password hashing and verification.
- **Activity 2.3:** Develop Forgot Password functionality allowing users to securely reset their passwords.
- **Activity 2.4:** Protect user data by validating authentication before accessing financial information and AI services.

Milestone 3: Financial Data Management

- **Activity 3.1:** Develop the Financial Details Form for collecting debt, income, EMI, interest rate, overdue months, and EMI payment information.
- **Activity 3.2:** Automatically calculate Total EMI Months and Remaining EMI Months using financial formulas.
- **Activity 3.3:** Store financial details into SQLite using SQLAlchemy ORM while maintaining financial history.
- **Activity 3.4:** Build APIs to retrieve the latest financial records and previous financial history.

Milestone 4: AI-Based Financial Assistance

- **Activity 4.1:** Integrate Google Gemini AI API for generating personalized debt settlement recommendations.
- **Activity 4.2:** Develop the Ask AI chatbot allowing users to ask financial questions and receive concise AI-generated responses.
- **Activity 4.3:** Implement AI-powered negotiation letter generation based on the user's financial condition.
- **Activity 4.4:** Format AI responses into short, readable points for improved user experience.

Milestone 5: Dashboard, Reports & Deployment

- **Activity 5.1:** Design an interactive dashboard displaying Financial Health Score, Risk Status, EMI Details, Debt-to-Income Analysis, and financial charts.
- **Activity 5.2:** Generate professional PDF reports containing financial summaries, AI recommendations, and negotiation letters.
- **Activity 5.3:** Develop the Financial History module to allow users to view previous financial records and generated AI reports.
- **Activity 5.4:** Test the application, validate API endpoints using FastAPI Swagger UI, and deploy the complete system locally with the React frontend and FastAPI backend.

Milestone 1: Model Selection and Architecture

In this milestone, we focus on establishing the development environment and designing the architecture required to build a secure, AI-powered debt relief and financial recovery platform.

Activity 1.1: Set Up the Development Environment

- **Create and activate a virtual environment:** Set up a Python virtual environment using **venv** to isolate project dependencies and ensure a consistent development environment.

```
dir
python -m venv venv
.\venv\Scripts\Activate.ps1
```

- **Install Dependencies:** Install all required libraries including **FastAPI, React.js, SQLAlchemy, SQLite, Google Gemini AI SDK, Axios, bcrypt, and Uvicorn** for backend services, frontend development, AI integration, and database operations.

```
uvicorn main:app
npm run dev
```

- **Start Backend:** Launch the FastAPI backend using **Uvicorn**, which starts the REST API server on **port 8000** and exposes interactive Swagger API documentation for testing.

```
cd backend
uvicon main:app --reload
```

- **Start Frontend:** Open a new terminal window, navigate to the frontend directory, install the required Node.js packages using **npm**, and execute **npm run dev** to launch the React application on **port 5173**.

```
cd frontend
npm run dev
```

Activity 1.2 & 1.3: AI Integration & System Configuration

The **Google Gemini AI API** was selected to provide intelligent financial recommendations, AI-powered financial chatbot responses, and professional debt negotiation letter generation. The AI service is configured once during application initialization using a secure API key, allowing the backend to efficiently process financial prompts and return personalized responses with minimal latency.

```
import google.generativeai as genai
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("gemini-2.5-flash")
```

Activity 1.4: Define Application Architecture

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the **React.js frontend**, **FastAPI backend**, **SQLite database**, and **Google Gemini AI integration**.
- **Detail Frontend Functionality:** Outline how users register, log in securely, enter financial information, view dashboards, generate AI recommendations, download PDF reports, and access financial history.
- **Outline Backend Responsibilities:** Specify how the FastAPI backend processes incoming API requests, validates input using **Pydantic**, performs financial calculations, communicates with the database, and integrates with Google Gemini AI.
- **Describe AI Integration Points:** Define how the backend sends user financial information to the **Google Gemini AI API** for generating personalized settlement recommendations, AI chatbot responses, and negotiation letters before returning structured results to the React frontend.

Milestone 2: Core Functionalities Development

Activity 2.1: Data Contract Validation

FastAPI uses Pydantic models to validate all incoming user data before processing. This ensures that financial information, login credentials, chatbot queries, and password reset requests follow the required schema, improving application reliability and preventing invalid data from entering the system.

```python
from pydantic import BaseModel
class LoginRequest(BaseModel):
    email: str
    password: str
class RegisterRequest(BaseModel):
    email: str
    password: str
class ResetPasswordRequest(BaseModel):
    email: str
    password: str
class FinancialRequest(BaseModel):
    user_id: int
    total_debt: int
    income: int
    emi: int
    interest_rate: float = 0
    overdue_months: int
    total_emi_months: int
    paid_emi_months: int
    remaining_emi_months: int
class ChatRequest(BaseModel):
    user_id: int
    question: str
class SettlementRequest(BaseModel):
    user_id: int
    history_id: int | None = None

    recommendation: str =""
    negotiation_letter: str = ""
```

Activity 2.2: Topic Generator Integration

The application integrates Google Gemini AI to generate personalized debt settlement recommendations, answer financial questions through an AI chatbot, and create professional negotiation letters. User inputs are converted into structured prompts, processed by Gemini AI, and the generated responses are returned to the frontend in real time.

```
        prompt = f"""
You are an expert financial advisor.

Answer ONLY the user's question.

Rules:
- Give exactly 10 numbered points.
- Each point must be short (10-15 words).
- Use simple English.
- Give practical advice.
- No introduction.
- No conclusion.
- No paragraphs.
- No markdown.

User Question:
{request.question}
"""

        response = model.generate_content(prompt)

        return {
            "reply": response.text
        }
```

Milestone 3: FastAPI Routing Orchestration

Milestone 3 focuses on developing and organizing the backend API endpoints that serve as the core communication layer between the React frontend, SQLite database, and Google Gemini AI. In this milestone, REST APIs are created to manage user authentication, financial information, AI-powered recommendations, chatbot interactions, negotiation letter generation, history management, and PDF report generation.

Activity 3.1: Writing the Main Routing Logic

- **Define API Endpoints:** Established multiple REST API routes using FastAPI decorators, including **/register**, **/login**, **/reset-password**, **/financial-details**, **/dashboard**, **/settlement-advice**, **/chat**, **/negotiation-letter**, **/download-report**, and **/history** to handle different application functionalities.
- **Service Orchestration:** The backend routes act as controllers by validating incoming requests using Pydantic models, retrieving and updating financial records from the SQLite database, communicating with the Google Gemini AI model, and returning structured JSON responses to the React frontend.
- **Automatic History Management:** Implemented financial history recording by automatically storing each submitted financial record, generated AI recommendation, and negotiation request, enabling users to review their previous financial analyses and reports.
- **Response Validation:** Each endpoint returns well-structured JSON responses containing success messages, user information, financial summaries, AI-generated recommendations, chatbot responses, and downloadable report data, ensuring reliable communication between the frontend and backend.

```python
@app.post("/register")
def register(data: RegisterRequest, db: Session = Depends(get_db)):

    # Check if email already exists
    existing_user = db.query(User).filter(User.email == data.email).first()

    if existing_user:
        return {
            "message": "Email already registered"
        }
    hashed_password = bcrypt.hashpw(
    data.password.encode("utf-8"),
    bcrypt.gensalt()
).decode("utf-8")
    # Create new user
    new_user = User(
        email=data.email,
        password=hashed_password,
        total_debt=0,
        income=0,
        emi=0,
        interest_rate=0,
        overdue_months=0,
        total_emi_months=0,
        paid_emi_months=0,
        remaining_emi_months=0,
    )

    db.add(new_user)
    db.commit()
    db.refresh(new_user)

    return {
        "message": "Account created successfully"
    }
```

```python
@app.post("/login")
def login(data: LoginRequest, db: Session = Depends(get_db)):
    print("Entered Email:", data.email)

    # Check if user exists
    user = db.query(User).filter(User.email == data.email).first(
    print("DB User:", user)

    if not user:
        return {
            "success": False,
            "message": "User not found. Please create an account.
        }
    print("DB Password:", user.password)
    print("Entered Password:", data.password)

    # Check password
    if not bcrypt.checkpw(
    data.password.encode("utf-8"),
    user.password.encode("utf-8")
):
        return {
            "success": False,
            "message": "Incorrect password."
    }

    # Login successful
    return {
        "success": True,
        "user_id": user.id,
        "message": "Login successful"
    }
```

```python
@app.get("/dashboard/{user_id}")
def dashboard(user_id: int, db: Session = Depends(get_db)):

    user = db.query(User).filter(User.id == user_id).first()

    if not user:
        return {"error": "User not found"}

    # Calculate EMI Ratio
    emi_ratio = (user.emi / user.income) * 100 if user.income > 0 else 100

# Calculate Debt-to-Income Ratio
    debt_ratio = user.total_debt / user.income if user.income > 0 else 999

    score = 100

# EMI Impact
    if emi_ratio >= 60:
       score -= 30
    elif emi_ratio >= 40:
       score -= 20
    elif emi_ratio >= 20:
       score -= 10

# Overdue Impact
    if user.overdue_months >= 12:
        score -= 30
    elif user.overdue_months >= 6:
        score -= 20
    elif user.overdue_months >= 3:
        score -= 10

# Debt Impact
    if debt_ratio >= 12:
        score -= 20
    elif debt_ratio >= 6:
        score -= 10

# Keep score between 0 and 100
    score = max(0, min(score, 100))

# Financial Status
    if score >= 80:
       status = "Low Risk 🟢"
    elif score >= 50:
       status = "Medium Risk 🟡"
    else:
        status = "High Risk 🔴"

    return {
    "user_id": user.id,
    "total_debt": user.total_debt,
    "income": user.income,
    "emi": user.emi,
    "overdue_months": user.overdue_months,
    "total_emi_months": user.total_emi_months,
     "paid_emi_months": user.paid_emi_months,
"remaining_emi_months": user.remaining_emi_months,
    "status": status,
    "score" : score,
    "message": "AI analysis completed"
}
```

```python
@app.post("/chat")
def chat(request: ChatRequest, db: Session = Depends(get_db)):
    try:
        # Get user from database
        user = db.query(User).filter(User.id == request.user_id).f

        if not user:
            return {
                "reply": "User not found."
            }

        # Build a personalized prompt
        prompt = f"""
You are an expert financial advisor.

Answer ONLY the user's question.

Rules:
- Give exactly 10 numbered points.
- Each point must be short (10-15 words).
- Use simple English.
- Give practical advice.
- No introduction.
- No conclusion.
- No paragraphs.
- No markdown.

User Question:
{request.question}
"""

        response = model.generate_content(prompt)

        return {
            "reply": response.text
        }

    except Exception as e:
        return {
            "reply": f"Error: {str(e)}"
        }
```

```python
@app.post("/financial-details")
def save_financial_details(data: FinancialRequest, db: Session = Depends(get_db)):

    user = db.query(User).filter(User.id == data.user_id).first()

    if not user:
        return {"message": "User not found"}

    user.total_debt = data.total_debt
    user.income = data.income
    user.emi = data.emi
    user.interest_rate = data.interest_rate
    user.overdue_months = data.overdue_months
    user.total_emi_months = data.total_emi_months
    user.paid_emi_months = data.paid_emi_months
    user.remaining_emi_months = data.remaining_emi_months

# Calculate Total EMI Months
    if data.interest_rate <= 0:
       total_emi_months = math.ceil(data.total_debt / data.emi)
    else:
       P = data.total_debt
       r = data.interest_rate / (12 * 100)
       EMI = data.emi

       total_emi_months = math.ceil(
        -math.log(1 - (P * r / EMI)) / math.log(1 + r)
    )

    user.total_emi_months = total_emi_months
    user.remaining_emi_months = max(
    0,
    total_emi_months - data.paid_emi_months
)

    history = FinancialHistory(
        user_id=user.id,
        total_debt=user.total_debt,
        income=user.income,
        emi=user.emi,
        interest_rate=user.interest_rate,
        overdue_months=user.overdue_months,
        total_emi_months=user.total_emi_months,
        paid_emi_months=user.paid_emi_months,
        remaining_emi_months=user.remaining_emi_months,
)

    db.add(history)

    db.commit()

    return {
    "message": "Financial details saved successfully"
}
```

```python
@app.post("/settlement-advice")
def settlement_advice(data: SettlementRequest, db: Session = Depends(get_db)):

    if data.history_id:

       user = (
        db.query(FinancialHistory)
        .filter(FinancialHistory.id == data.history_id)
        .first()
    )

    else:

       user = (
        db.query(User)
        .filter(User.id == data.user_id)
        .first()
    )

    if not user:
        return {"error": "User not found"}
    # Calculate EMI Ratio
    emi_ratio = (user.emi / user.income) * 100 if user.income > 0 else 100

# Calculate Debt-to-Income Ratio
    debt_ratio = user.total_debt / user.income if user.income > 0 else 999

    score = 100

# EMI Impact
    if emi_ratio >= 60:
       score -= 30
    elif emi_ratio >= 40:
       score -= 20
    elif emi_ratio >= 20:
       score -= 10

# Overdue Impact
    if user.overdue_months >= 12:
        score -= 30
    elif user.overdue_months >= 6:
        score -= 20
    elif user.overdue_months >= 3:
        score -= 10

# Debt Impact
    if debt_ratio >= 12:
        score -= 20
    elif debt_ratio >= 6:
        score -= 10

# Keep score between 0 and 100
    score = max(0, min(score, 100))

# Financial Status
    if score >= 80:
       status = "Low Risk 🟢"
    elif score >= 50:
       status = "Medium Risk 🟡"
    else:
       status = "High Risk 🔴"
```

```python
@app.post("/negotiation-letter")
def negotiation_letter(
    data: SettlementRequest,
    db: Session = Depends(get_db)
):
    if data.history_id:
        user = (
            db.query(FinancialHistory)
            .filter(FinancialHistory.id == data.history_id)
            .first()
        )
    else:
        user = (
            db.query(User)
            .filter(User.id == data.user_id)
            .first()
        )

    if not user:
        return {"error": "User not found"}

    prompt = f"""
Generate a professional loan settlement request letter.

Borrower's Details:
Total Debt: ₹{user.total_debt}
Income: ₹{user.income}
Monthly EMI: ₹{user.emi}
Overdue Months: {user.overdue_months}

Write a professional negotiation letter requesting
loan settlement or EMI restructuring.
"""

    letter = f"""
Dear Loan Manager,

Subject: Request for Loan Settlement / EMI Restructuring

Loan Account Number: _____

I am writing to respectfully request a review of my current loan account.

Due to financial constraints, I kindly request the bank to consider eithe

I assure you of my sincere intention to cooperate with the bank and reso

Thank you for your consideration.

Sincerely,
```

```python
@app.get("/download-negotiation-letter")
def download_negotiation_letter():

    pdf_path = "Negotiation_Letter.pdf"

    return FileResponse(
        path=pdf_path,
        filename="Negotiation_Letter.pdf",
        media_type="application/pdf"
    )


# ============================================================
# DOWNLOAD REPORT
# ============================================================

@app.post("/download-report")
def download_report(
    data: SettlementRequest,
    db: Session = Depends(get_db)
):

    if data.history_id:
        user = (
            db.query(FinancialHistory)
            .filter(FinancialHistory.id == data.history_id)
            .first()
        )
    else:
        user = (
            db.query(User)
            .filter(User.id == data.user_id)
            .first()
        )

    if not user:
        return {"error": "User not found"}

    pdf_path = "FinRelief_Report.pdf"

    doc = SimpleDocTemplate(pdf_path)

    story = []

    styles = getSampleStyleSheet()

    title_style = styles["Heading1"]
    title_style.alignment = TA_CENTER
    title_style.textColor = HexColor("#2563EB")

    heading_style = styles["Heading2"]
    normal_style = styles["BodyText"]
```

```python
@app.post("/reset-password")
def reset_password(
    data: ResetPasswordRequest,
    db: Session = Depends(get_db)
):

    user = db.query(User).filter(
        User.email == data.email
    ).first()

    if not user:
        return {
            "success": False,
            "message": "User not found."
        }

    hashed_password = bcrypt.hashpw(
        data.password.encode("utf-8"),
        bcrypt.gensalt()
    ).decode("utf-8")

    user.password = hashed_password

    db.commit()

    return {
        "success": True,
        "message": "Password updated successfully."
    }
```

```python
@app.get("/history/{user_id}")
def get_history(user_id: int, db: Session = Depends(get_db)):

    history = (
        db.query(FinancialHistory)
        .filter(FinancialHistory.user_id == user_id)
        .order_by(FinancialHistory.created_at.desc())
        .all()
    )

    return history
@app.get("/history-details/{history_id}")
def get_history_details(history_id: int, db: Session = Depends(get_db)):

    history = (
        db.query(FinancialHistory)
        .filter(FinancialHistory.id == history_id)
        .first()
    )

    if not history:
        return {"message": "History not found"}

    # EMI Ratio
    emi_ratio = (
        (history.emi / history.income) * 100
        if history.income > 0 else 100
    )

    # Debt Ratio
    debt_ratio = (
        history.total_debt / history.income
        if history.income > 0 else 999
    )

    score = 100

    # EMI Impact
    if emi_ratio >= 60:
        score -= 30
    elif emi_ratio >= 40:
        score -= 20
    elif emi_ratio >= 20:
        score -= 10

    # Overdue Impact
    if history.overdue_months >= 12:
        score -= 30
    elif history.overdue_months >= 6:
        score -= 20
    elif history.overdue_months >= 3:
        score -= 10

    # Debt Impact
    if debt_ratio >= 12:
        score -= 20
    elif debt_ratio >= 6:
        score -= 10

    score = max(0, min(score, 100))
```

Activity 3.2: Application Entry Point & Infrastructure Configuration

- **Configure main.py:** Created the FastAPI application instance and configured the backend as the central service responsible for handling authentication, financial analysis, AI services, and report generation.
- **Configure Middleware & Database:** Enabled Cross-Origin Resource Sharing (CORS) to allow secure communication with the React frontend and configured the SQLite database using SQLAlchemy ORM for persistent data storage.
- **Application Services:** Integrated Google Gemini AI, database sessions, password encryption using bcrypt, PDF generation modules, and financial history management into the backend application workflow.
- **Swagger UI Generation:** Enabled automatic OpenAPI 3.0 documentation through **/docs**, allowing developers to interactively test and validate all backend REST API endpoints and request schemas.

```python
app = FastAPI()

# CORS (React connection)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# -------------------------
# DATABASE SESSION
# -------------------------
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

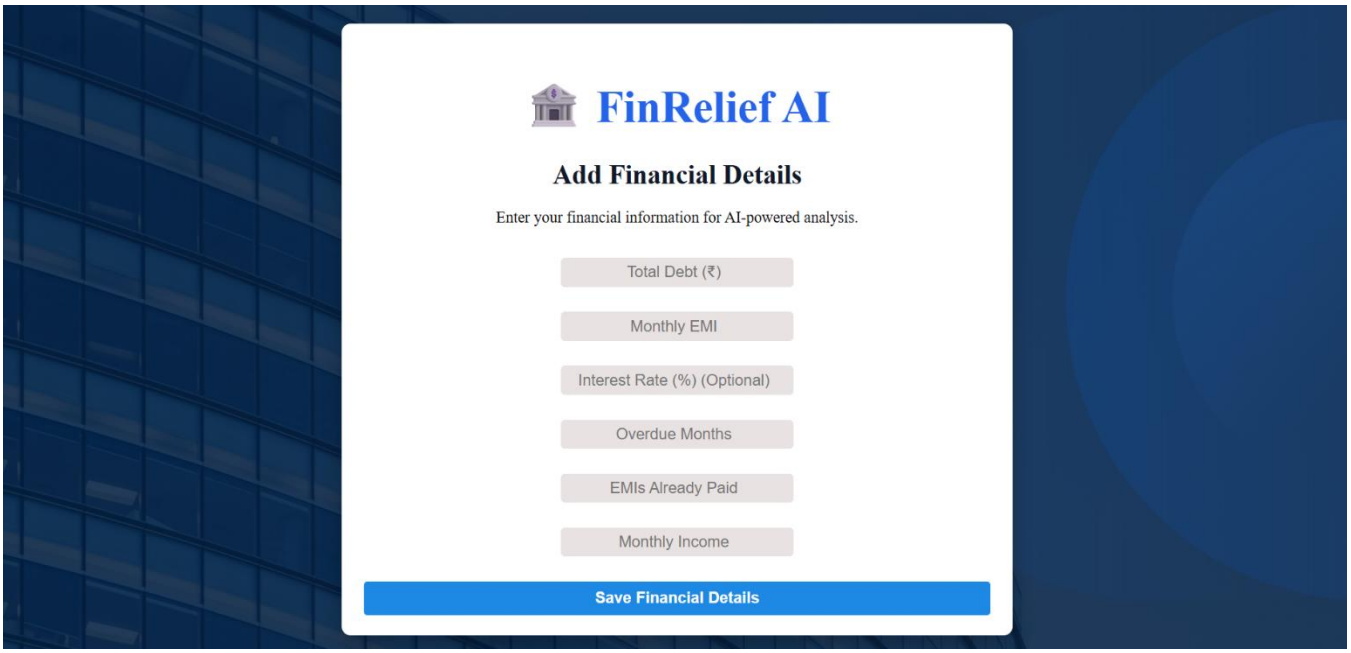
```python
from database import SessionLocal, User, FinancialHistory

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel("gemini-2.5-flash")

app = FastAPI()
```

Milestone 4: Frontend Development using Streamlit

Milestone 4 focuses on developing a modern and user-friendly web interface using React.js. The frontend provides seamless navigation between pages, validates user inputs, communicates with the FastAPI backend through REST APIs, and dynamically renders AI-generated financial insights without requiring full page reloads.

Activity 4.1: Designing the User Interface

The frontend was developed using React.js by creating reusable components and individual pages such as **Login**, **Register**, **Forgot Password**, **Home**, **Financial Details**, **Dashboard**, and **History**. Responsive forms, navigation, and interactive buttons were designed to provide a smooth and intuitive user experience while collecting financial information securely.



Activity 4.2: Dynamic Content Rendering and State Management

React Hooks, including **useState** and **useEffect**, were used to manage application state and dynamically update the user interface. Axios was integrated to communicate with the FastAPI backend, enabling real-time display of AI-generated financial recommendations, chatbot responses, negotiation letters, financial history, dashboard analytics, and downloadable PDF reports without refreshing the page.

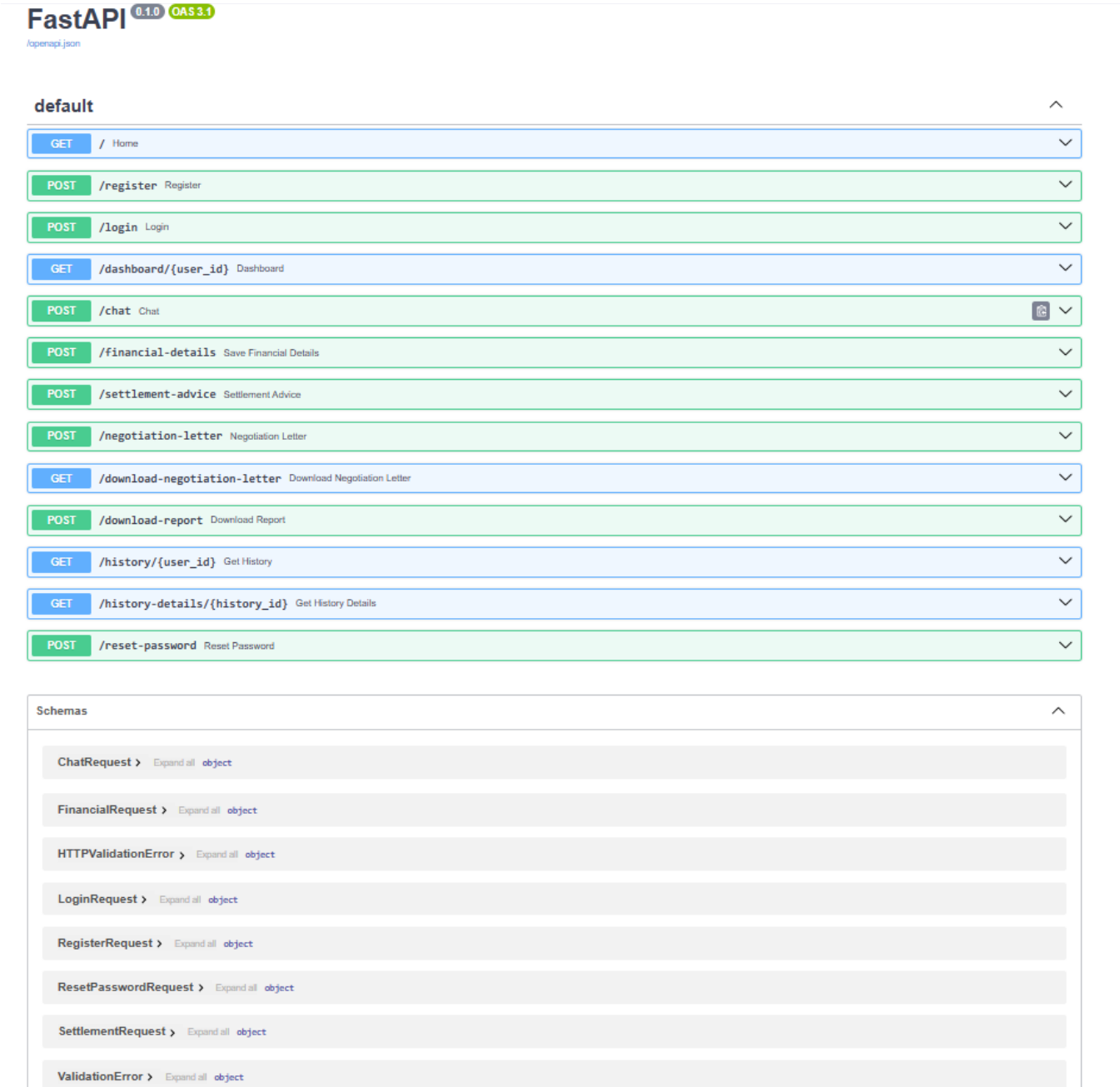
```
const [reply, setReply] = useState("");
const [aiAdvice, setAiAdvice] = useState("");
const sendMessage = async () => {
  const res = await axios.post("http://127.0.0.1:8000/chat", {
    user_id: Number(userId),
    question: message,
  });
  setReply(res.data.reply);
};
```

```
const getSettlementAdvice = async () => {
try {
 const res = await axios.post(
"http://127.0.0.1:8000/settlement-advice",
{
  user_id: Number(userId),
  history_id: historyId ? Number(historyId) : null,
}
);
```

Milestone 5: Testing and Deployment

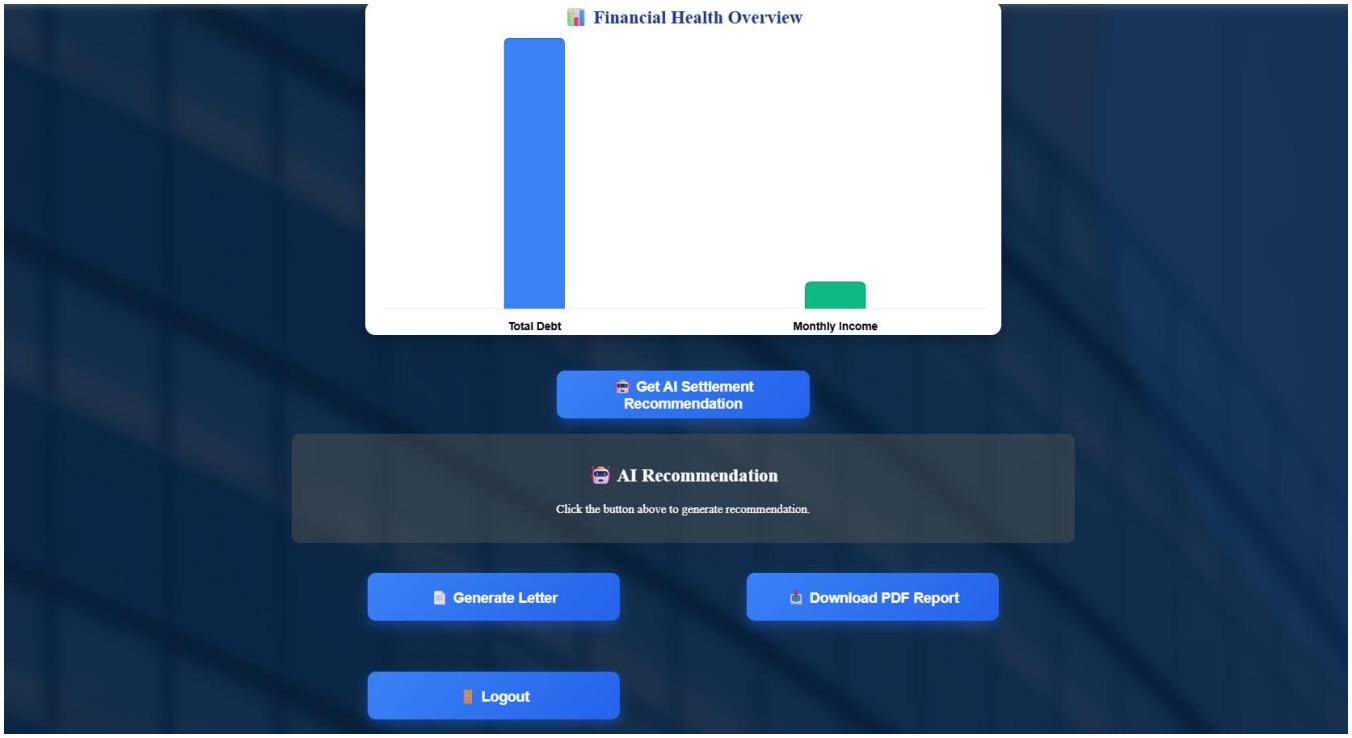
Activity 5.1: Functional Testing

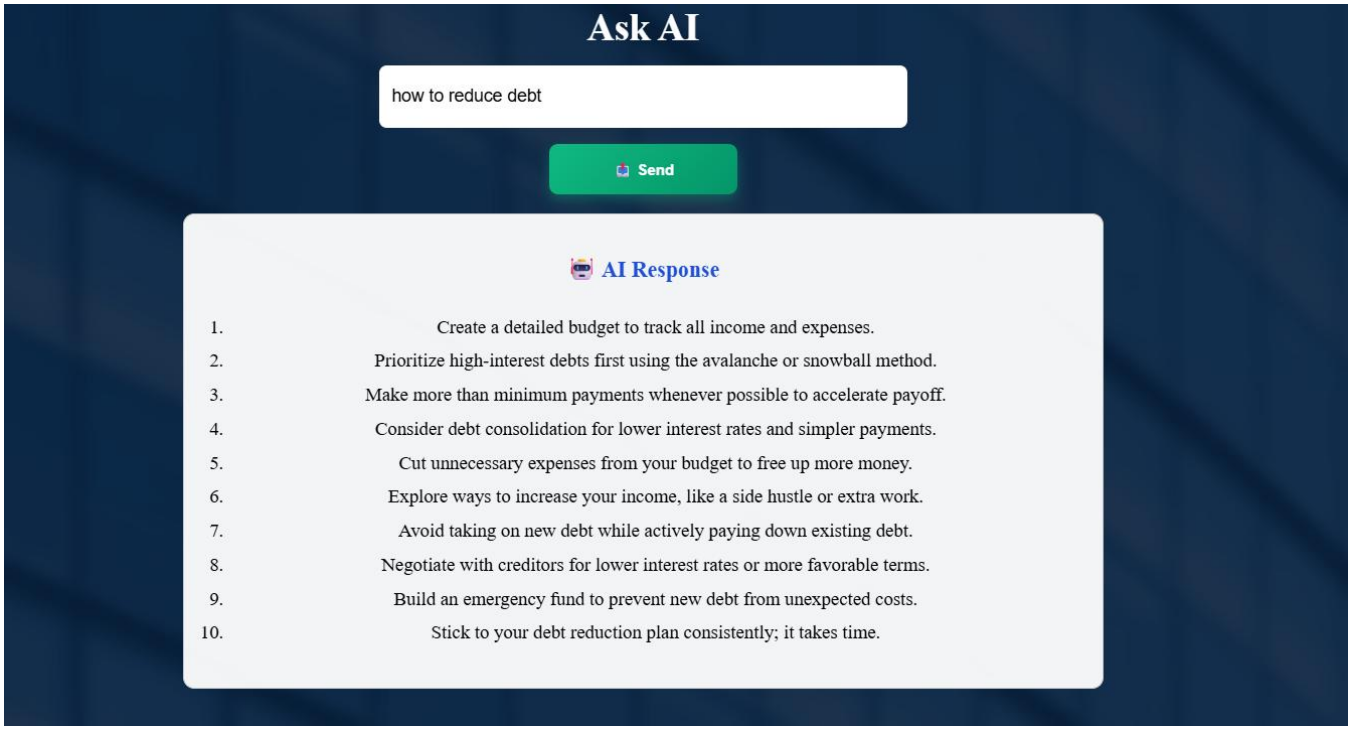
The application was functionally tested by running the FastAPI backend and React frontend, followed by validating all major features through the Swagger UI and the web interface. User registration, login, password reset, financial data submission, AI recommendation generation, chatbot interaction, negotiation letter generation, history retrieval, and PDF report download were tested to ensure correct functionality.



Activity 5.2: Application Testing

The application was tested by executing different user workflows through the React interface. Various financial scenarios were entered to verify dashboard calculations, AI recommendations, chatbot responses, negotiation letter generation, financial history, and PDF report creation. The system produced accurate and consistent results across all tested scenarios.





FinRelief AI

AI Powered Debt Relief and Financial Recovery System

Financial Report

User ID : 6

Financial Summary

Total Debt : Rs. 500000
Monthly Income : Rs. 50000
Monthly EMI : Rs. 10000
Interest Rate : 0%
Overdue Months : 0
EMIs Already Paid : 2
Total EMI Months : 50
Remaining EMI Months : 48

AI Financial Health

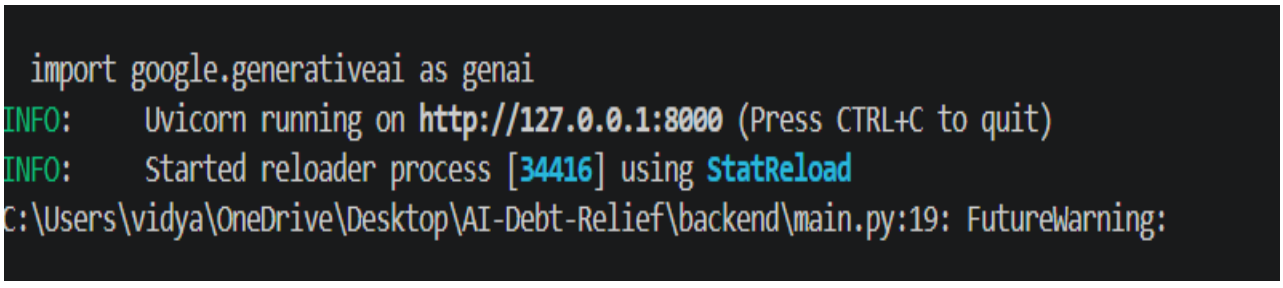
Financial Health Score : 80%
AI Status : Low Risk ■

AI Settlement Recommendation

AI Negotiation Letter

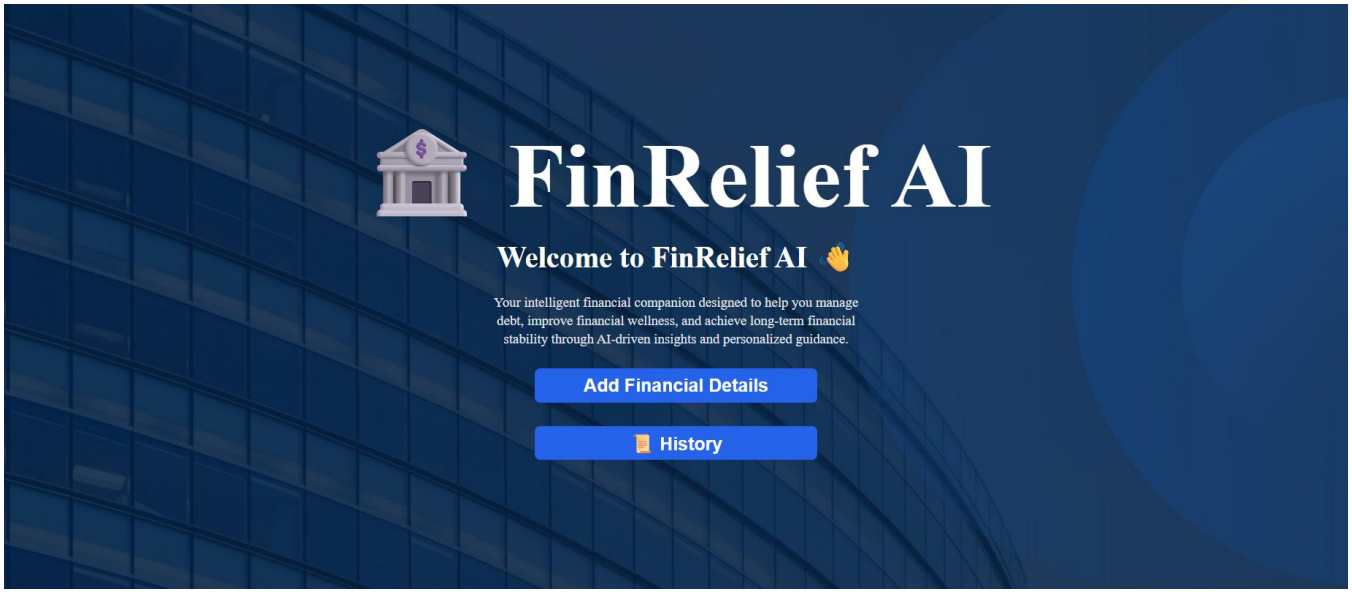
Activity 5.3: Application Boot Sequence

The application follows a client-server architecture where the FastAPI backend and React frontend run simultaneously. The backend provides REST APIs on port 8000, while the React frontend is served through the Vite development server on port 5173. Both services communicate using HTTP requests through Axios.

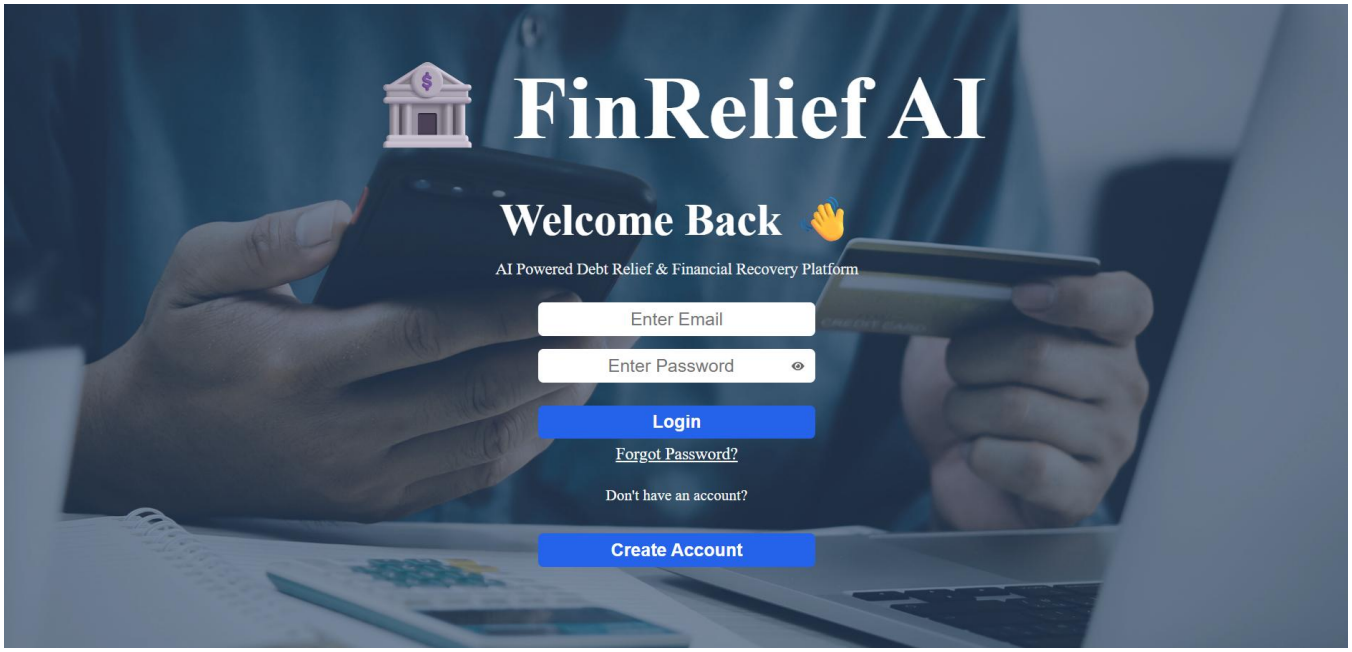


Exploring the Website's Web Pages:

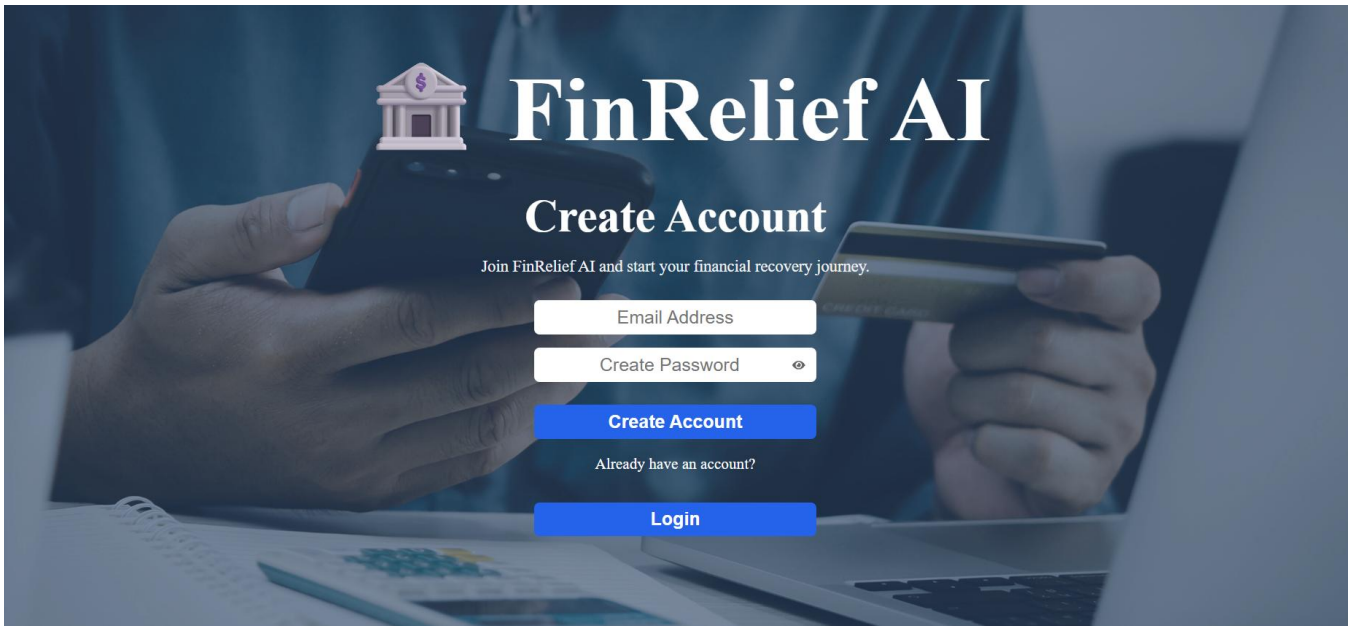
Home Page:



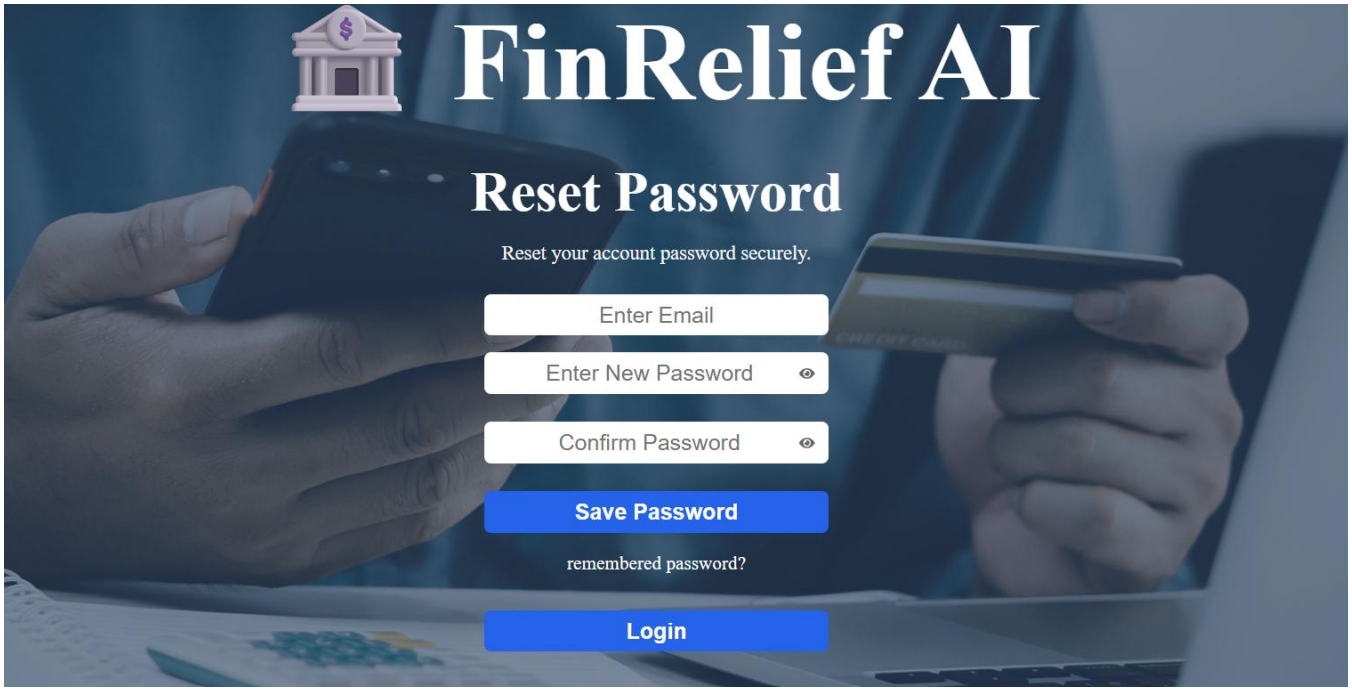
Login Page:



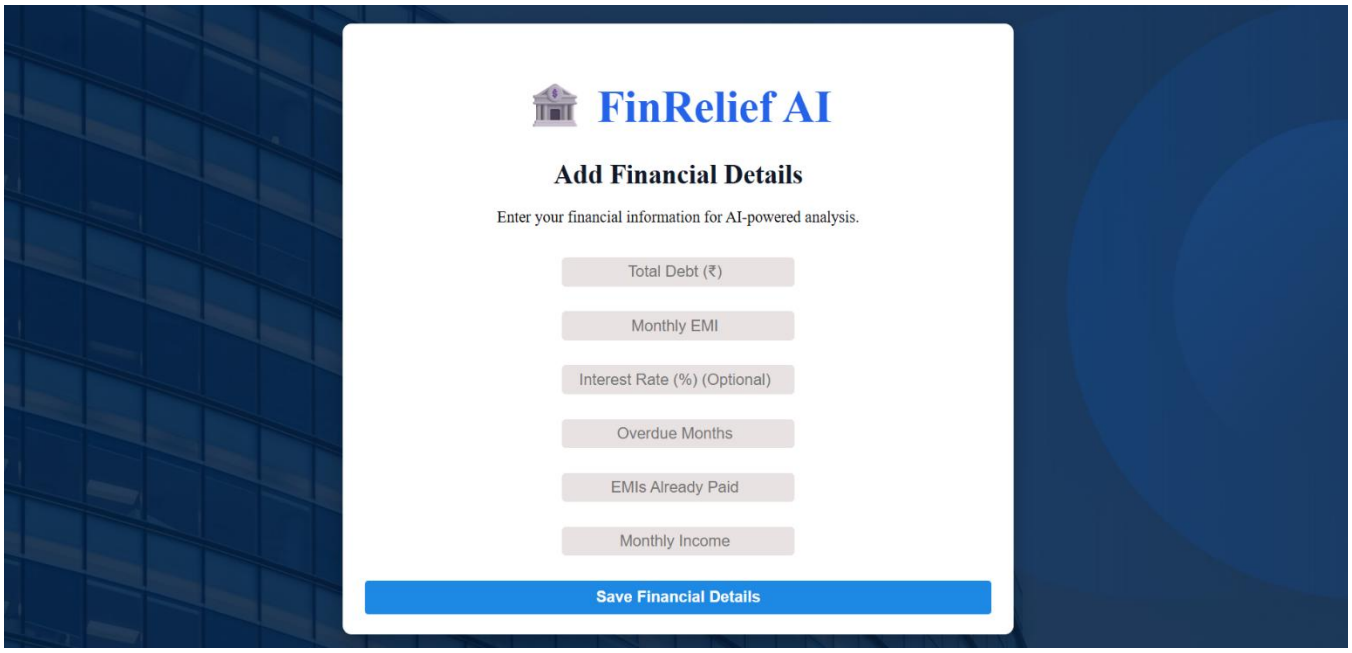
Register Page:



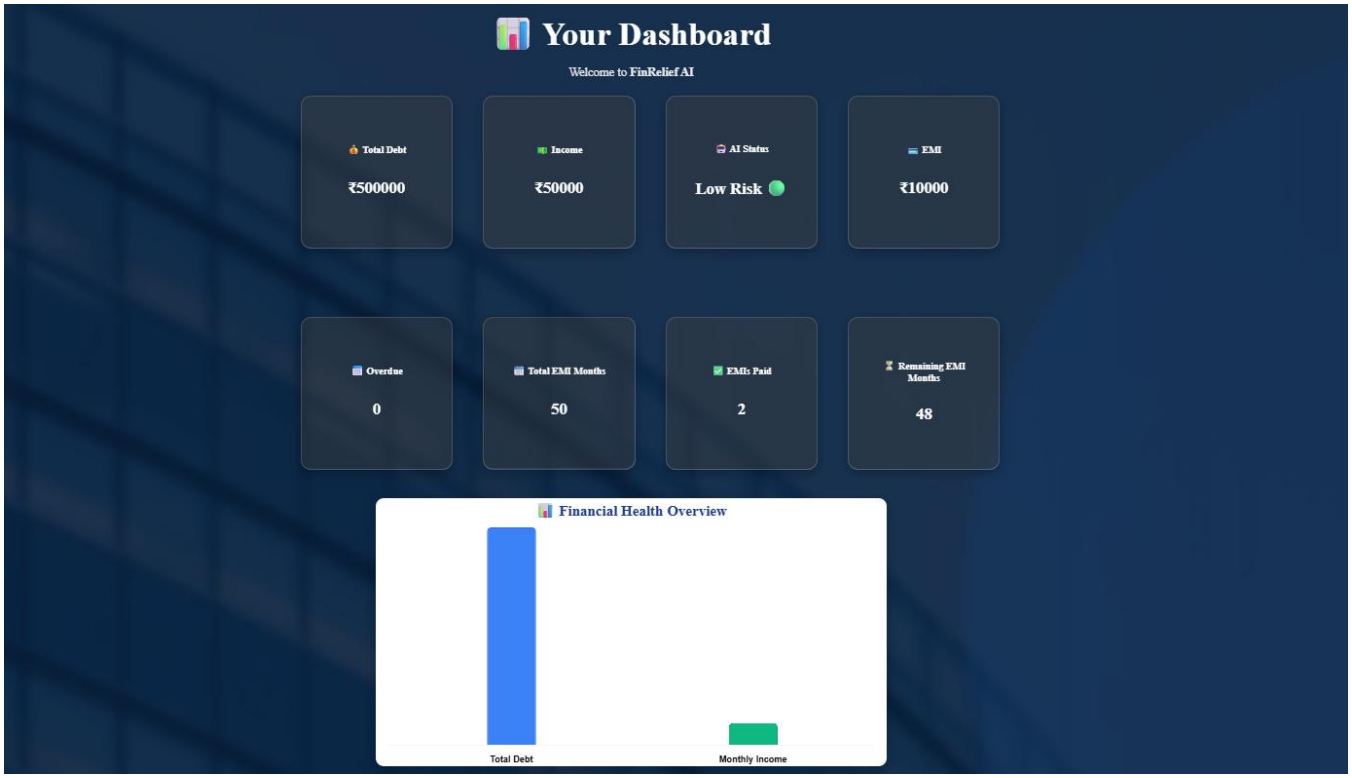
ForgotPassword Page:



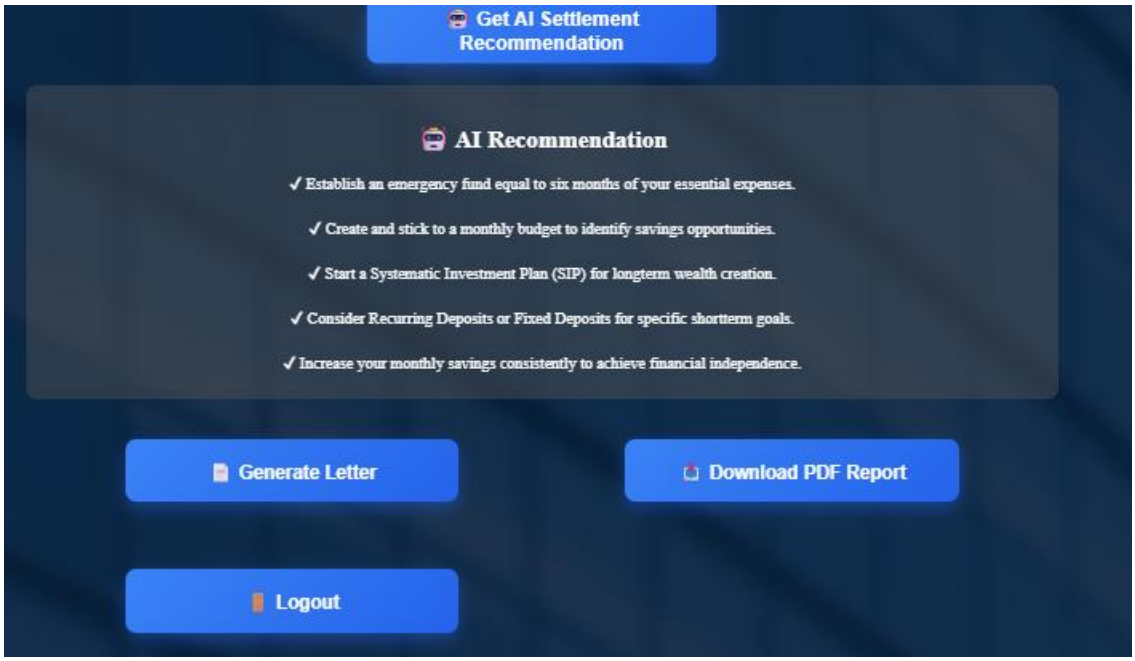
FinancialDetails Form:


FinRelief AI
Add Financial Details
Enter your financial information for AI-powered analysis.
Total Debt (₹)
Monthly EMI
Interest Rate (%) (Optional)
Overdue Months
EMIs Already Paid
Monthly Income
Save Financial Details

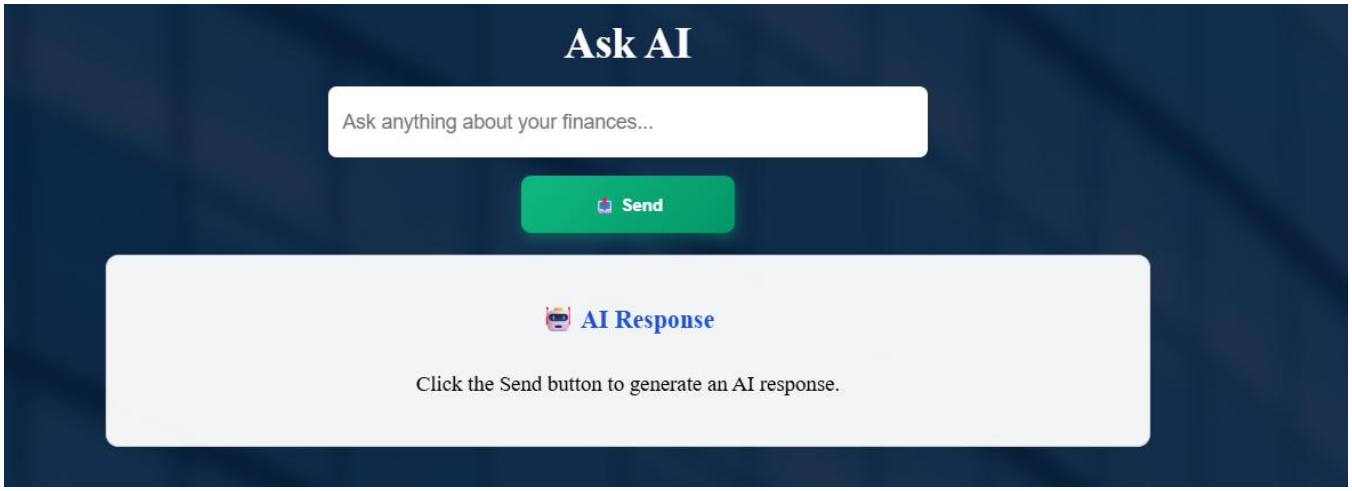
Dashboard:


Your Dashboard
Welcome to FinRelief AI
Total Debt
₹500000
Income
₹50000
AI Status
Low Risk
EMI
₹10000
Overdue
0
Total EMI Months
50
EMIs Paid
2
Remaining EMI Months
48
Financial Health Overview
Total Debt
Monthly Income

AI Recommendation Section:



Ask AI Chatbot:



Negotiation Letter:

FinRelief AI

AI Powered Debt Relief and Financial Recovery System

AI Negotiation Letter

Dear Loan Manager,

Subject: Request for Loan Settlement / EMI Restructuring

Loan Account Number: _____

I am writing to respectfully request a review of my current loan account. My total outstanding debt is Rs. 300000, my monthly income is Rs. 50000, and my current monthly EMI is Rs. 10000. I have 0 overdue month(s).

Due to financial constraints, I kindly request the bank to consider either a suitable loan settlement or a restructuring of my EMI schedule. I am fully committed to repaying my obligations and would greatly appreciate a repayment plan that better matches my current financial capacity.

I assure you of my sincere intention to cooperate with the bank and resolve this matter responsibly. I request you to review my financial situation and suggest the most suitable repayment option.

Thank you for your consideration.

Sincerely,

Borrower

Signature: _____

Date: _____

History Page:

PDF Report:

FinRelief AI

AI Powered Debt Relief and Financial Recovery System

Financial Report

User ID : 6

Financial Summary

Total Debt : Rs. 500000
Monthly Income : Rs. 50000
Monthly EMI : Rs. 10000
Interest Rate : 0%
Overdue Months : 0
EMIs Already Paid : 2
Total EMI Months : 50
Remaining EMI Months : 48

AI Financial Health

Financial Health Score : 80%
AI Status : Low Risk ■

AI Settlement Recommendation

AI Negotiation Letter

Conclusion

FinRelief AI – AI Powered Debt Relief & Financial Recovery Platform is an intelligent financial assistance system designed to help individuals effectively manage their debt and improve their financial well-being. Developed using **React.js**, **FastAPI**, **Python**, **SQLite**, **SQLAlchemy ORM**, and **Google Gemini AI**, the platform combines modern web technologies with Artificial Intelligence to provide users with personalized financial guidance. By analyzing user financial data, the system generates AI-powered debt settlement recommendations, answers financial queries through an interactive chatbot, creates professional negotiation letters, and presents financial insights through a comprehensive dashboard.

The application emphasizes **security**, **usability**, and **performance** by implementing secure user authentication, password hashing using **bcrypt**, strong input validation, RESTful API communication, and efficient database management. Features such as financial history tracking, PDF report generation, and real-time AI assistance provide users with a complete financial management experience while simplifying complex debt-related decision-making.

From a software engineering perspective, the project follows a modular architecture that separates the frontend, backend, AI services, and database layers, making the application scalable, maintainable, and easy to extend. The use of FastAPI with Pydantic models ensures reliable data validation, while React.js provides a responsive and interactive user interface that enhances the overall user experience.

Looking ahead, the platform offers significant opportunities for future enhancement. Features such as **email-based OTP verification for password recovery**, **multi-factor authentication**, **loan eligibility prediction using machine learning**, **credit score analysis**, **bank API integration**, **real-time financial notifications**, **cloud database deployment**, **Docker containerization**, and **deployment on cloud platforms such as AWS, Azure, or Google Cloud** can further improve the application's scalability, security, and practical usability. With these future enhancements, FinRelief AI has the potential to evolve into a comprehensive AI-driven financial advisory platform that empowers users to make informed financial decisions, reduce debt effectively, and achieve long-term financial stability.